
A BRAVE NEW CURRICULUM - SOLUCIÓN GUÍA

Universidad de Bogotá Jorge Tadeo Lozano

Alvarado Becerra Ludwig - Franco Calderón Alejandro
Estructuras de datos - 2026-1S

Índice

1. Interpretación del problema, entrada y salida de datos	1
1.1. Objetivo	1
1.2. Entradad de datos esperada	1
1.3. Salida de datos esperada	1
2. Solución conceptual	1
3. Código modelo	2
3.1. Lectura de datos	2
3.2. Creación del stack	2
3.3. Recorrer el string de entrada	3
3.4. Validar paréntesis de apertura	3
3.5. Comprobar paréntesis de cierre	3
3.6. Validar stack vacío	4
3.7. Código completo	4

1. Interpretación del problema, entrada y salida de datos

1.1. Objetivo

Dada una cadena de paréntesis, verificar si esta es valida comprobando el emparejamiento de los paréntesis.

1.2. Entradad de datos esperada

Se nos diferentes cadenas de paréntesis en cada línea enviada al programa, el programa debe terminar cuando se encuentra una línea que sea 0

```
((()))  
{({[[[({})]]])}}}  
[]{()  
{}  
0
```

Algorítmicamente, se debe parar el algoritmo cuando la entrada sea '0'.

1.3. Salida de datos esperada

Por cada cadena de paréntesis, imprimir YES en caso de que la cadena sea valida, si no lo es, imprimir NO. Tener en cuenta que la salida debe ser en mayúsculas.

2. Solución conceptual

En el contexto de un compilador, es fundamental verificar que los paréntesis, corchetes y llaves estén correctamente balanceados. Es decir:

- Cada símbolo de apertura debe tener su correspondiente cierre.
- El orden de cierre debe ser correcto.

La idea clave es pensar en que **el último paréntesis debe ser el primero en cerrarse**. Esta idea encaja con el principio de funcionamiento de un *stack* (pila), donde el último elemento que ingresa es el primero en salir. Las operaciones básicas de un *stack* incluyen:

- `push(x)`, guardar `x` en la pila.
- `is_empty()`, retorna `true` si no hay elementos en la pila, en caso contrario devuelve `false`.
- `pop()`, saca (y retorna) el último elemento que fue guardado en la pila.

Lo ideal es tener dos estructuras de datos para guardar la información:

- `string` para guardar la entrada de datos.

- **stack**, usado para almacenar los paréntesis abiertos que se encuentren.

Teniendo en cuenta la lógica de programación, los pasos a seguir con el algoritmo¹ es:

1. Recorrer el **string** de izquierda a derecha.
2. Si se encuentra un símbolo de apertura (, {, [. Entonces, agregar al **stack** usando **push()**.
3. Si se encuentra un símbolo de cierre), },]:
 - Si la pila está vacía, entonces imprimir **N0**.
 - Usar **pop()** y verificar si coincide con el paréntesis de apertura, si sí, seguir, si no, imprimir **N0**.
4. Después de haber iterado sobre todo el **string**, verificar si está vacío el **stack**, si sí, imprimir **YES**, de lo contrario, imprimir **N0**.

3. Código modelo

Se invita a que el estudiante intente implementar la solución únicamente con el algoritmo descrito en los pasos antes de ver el código modelo de la solución.

3.1. Lectura de datos

Se debe iniciar el programa leyendo datos como tipo **str** y guardar la información dentro de una variable **s**. Esta acción se debe repetir hasta que en algún momento **s** sea 0, en este momento el programa se debe de detener. Para hacer la acción de repetir indefinidamente una serie de acciones mientras se cumple una condición, se puede utilizar un ciclo tipo **while**

```
1 s = input()
2
3 while (s != '0'):
4     flag = True
5     ...
6     s = input()
```

Donde ... es la lógica que se implementará en próximos pasos.

A su vez, se utiliza una variable **flag** que se utilizará para validar imprimir luego la respuesta, esta va a ser únicamente modificada si encuentra algo invalido.

3.2. Creación del stack

Hay múltiples maneras de implementar el stack que se necesita para el problema:

- Usando la clase **deque** del modulo **collections**
- Implementar desde cero un modulo **stack** con sus clases de nodo y stack.

¹Se recomienda leer el algoritmo en pasos e intentar pasarlo a código antes de ver la solución del código modelo

- Usar la estructura ya definida en python de `list` y sus métodos de:
 - `append(x)` para agregar elementos al final de la lista.
 - `pop()` eliminar el último elemento de la lista.

Por simplicidad se va a utilizar `list`, tomar en cuenta que esto no es un *stack* puramente, sino más bien, se utiliza como una interpretación de la misma.

Para crearlo, simplemente se declara una variable `stack` como una lista vacía.

```
1 stack = []
```

3.3. Recorrer el string de entrada

Una de las claves de este ejercicio es reconocer el *string* del *stack*. Para verificar la validez del *stack* se debe **recorrer** el *string*, esto se puede realizar con un ciclo `for`, una posible manera es iterando sobre los índices de la cadena:

```
1 for i in range(len(s)):
```

Para este ciclo, los posibles valores de la variable i van a ser $\{0, 1, 2, \dots, |s| - 1\}$. Es decir, desde 0 hasta la longitud de la cadena menos 1.

3.4. Validar paréntesis de apertura

Teniendo en cuenta la solución conceptual, cada vez que se encuentre cualquier tipo de paréntesis abierto, entonces, este se debe agregar al *stack*. Es decir, si mientras recorremos la cadena encontramos un caracter de apertura (, [o {, entonces agregar el caracter al *stack*.

```
1 if (s[i] == '(' or s[i] == '[' or s[i] == '{'):  
2     stack.append(s[i])
```

3.5. Comprobar paréntesis de cierre

Se sigue también la solución conceptual para este punto.

Si encontramos un paréntesis de cierre y el *stack* está vacío, entonces cambiar `flag` a `False`.

Si encontramos un paréntesis de cierre y el *top* del *stack* hace **match**, entonces remover el *top* del *stack*. De otra forma, cambiar `flag` a `False`.

```
1 if (not len(stack)):  
2     flag = False  
3     break  
4 elif (s[i] == ')') and stack[-1] == '(':  
5     stack.pop()  
6 elif (s[i] == ']') and stack[-1] == '[':  
7     stack.pop()  
8 elif (s[i] == '}') and stack[-1] == '{':  
9     stack.pop()
```

```
10 else:
11     flag = False
12     break
```

3.6. Validar stack vacío

Si después de recorrer toda la cadena el *stack* está vacío, imprima YES, de lo contrario, imprimir NO. A su vez, recordar utilizar `flag` para verificar si hubo un cambio de esta variable cuando se terminó el ciclo.

```
1 if (flag and not len(stack)):
2     print("YES")
3 else:
4     print("NO")
```

Recordar que la función `len()` retorna un número que puede ser interpretado como `False` si es 0 y `True` si es > 0 .

3.7. Código completo

```
1 s = input()
2
3 while (s != '0'):
4     stack = []
5     flag = True
6
7     for i in range(len(s)):
8         if (s[i] == '(' or s[i] == '[' or s[i] == '{'):
9             stack.append(s[i])
10        elif (not len(stack)):
11            flag = False
12            break
13        elif (s[i] == ')' and stack[-1] == '('):
14            stack.pop()
15        elif (s[i] == ']' and stack[-1] == '['):
16            stack.pop()
17        elif (s[i] == '}' and stack[-1] == '{'):
18            stack.pop()
19        else:
20            flag = False
21            break
22
23 if (flag and not len(stack)):
24     print("YES")
25 else:
26     print("NO")
```

27

28

```
s = input()
```



UTADEO

UNIVERSIDAD DE BOGOTÁ JORGE TADEO LOZANO